Assignment N°1 - Exam

1. What problem are you solving and why does it matter?

Chile has a housing crisis. There is a deficit of 650,000 housing units, and one of the primary structural causes is that building permits take an average of 225 days to be approved - against a legal limit of 30 days. That is a 7.5x overrun, and it adds 9.5–12% to the final price of every housing unit in the country.
The core bottleneck is not corruption or bad intent - it is information asymmetry. Architects submit plans with errors or omissions that violate the OGUC (the national building code) or the municipality's local zoning plan (PRC). The DOM reviewer issues an Acta de Observaciones - a list of compliance failures - and the clock resets. This loop repeats an average of 2.3 times per project, adding months to each cycle.
The problem matters because it is not just a bureaucratic inconvenience: every month of delay costs a developer USD 15,000–50,000 in financing costs, pushes housing prices higher, and deepens a crisis that is locking an entire generation out of homeownership. It is a solvable information problem sitting at the intersection of AI, law, and public policy - and no one has built the solution in Chile or anywhere in Latin America.

2. How do you plan to build with AI?

The product is a multi-agent pre-review system that ingests an architect's project parameters and checks them against the OGUC and the municipality's PRC before submission — generating a structured observation report in the same format as a real DOM Acta de Observaciones.

The system has four AI agents orchestrated as a directed graph using LangGraph:

Agent	Role	Tool
Input Parser Agent	Extracts structured project data (zone, surface, height, constructibility, etc.) from submitted forms	Claude 3.5 Sonnet
Regulatory Retriever Agent	Queries Supabase pgvector — a vector database of 986 pre-chunked OGUC + PRC articles — using semantic search	LangGraph + Supabase pgvector
Compliance Reasoner Agent	Reasons over retrieved articles and project parameters to identify violations, citing specific articles	Claude 3.5 Sonnet
Report Generator Agent	Formats findings into a numbered Acta de Observaciones — same structure as a real DOM document	Claude 3.5 Sonnet

What I delegate to AI: legal cross-referencing, natural language generation of observation text, semantic search over 986 regulatory chunks, structured output formatting, and memory management across sessions.
What I build manually: the chunking pipeline for the OGUC/PRC, the FastAPI backend, the React frontend, and the evaluation framework comparing AI-generated Actas against real ones.
Stack:

- **LLM:** Claude 3.5 Sonnet (Anthropic API) - best legal reasoning and structured output

- **Agent Orchestration:** LangGraph - native directed graph with state, streaming, and human-in-the-loop checkpoints
- **Memory & Persistence:** Supabase (PostgreSQL + pgvector) - stores agent state, conversation history, past project runs, and regulatory vectors in one place; LangGraph's native Supabase checkpointer handles this automatically
- **Auth:** Google OAuth via Supabase Auth — architects log in with Google, their project history and past compliance reports persist across sessions
- **Backend**: FastAPI - lightweight, async, LangGraph-compatible
- **Frontend:** React + Vite + Tailwind - generated by Claude Code, lives in the same repo as the backend

I will use Claude Cowork and Claude Code as the main agents for building this.

Prototype approach: Start with a single municipality (Las Condes), a single project type (residential Edificación Nueva), and a single zone (ZHR2). Validate against real Actas de Observaciones collected from public sources. Expand from there.

3. What do you hope to learn about AI through this project?

Four specific things:

1. First, how to productize an AI agent system. I have built multi-agent pipelines before, but I have never shipped one as a real product — with authentication, persistent user state, a production-grade frontend, and an evaluation framework. The gap between a working agent demo and a product someone actually trusts and uses is enormous. I want to learn what it takes to close that gap: how do you handle errors gracefully, communicate uncertainty to non-technical users, and design a UI that makes an AI agent feel reliable rather than opaque?
2. Second, how to design and implement memory for AI agents. Memory is the hardest unsolved problem in production agent systems. For this product, there are three distinct memory layers I need to get right: short-term (the agent's working state within a single compliance run), session-level (what the architect submitted last time, which observations were resolved), and long-term (patterns across hundreds of projects — which violations are most common in which zones, which article combinations appear together). I want to learn how LangGraph's checkpointer and Supabase's persistence layer handle each of these, and where the architecture breaks down at scale.
3. Third, how to build reliable RAG systems for high-stakes legal reasoning. The system must retrieve the right regulatory article, reason correctly about whether a specific project parameter violates it, and cite it accurately. Hallucination here has real consequences. I want to learn what retrieval strategies, chunking approaches, and prompt architectures actually work when the cost of being wrong is months of delay for an architect.
4. Fourth, how to design human-in-the-loop workflows for a regulated industry. The AI cannot be the final authority — it is a pre-review tool, not a replacement for the DOM. I want to learn how to design the right handoff points using LangGraph's interrupt mechanism: when does the system pause and require human confirmation, how do you communicate confidence levels to a non-technical user, and how do you build trust incrementally so that users rely on the tool without over-relying on it?

Planning:

- Total time budget: 12–15 hours
- Deadline: April 16th
- Goal: A working prototype that can ingest a real project's parameters, check them against the OGUC + Las Condes PRC, and generate a structured Acta de Observaciones

- No customer discovery — validate against real documents instead of interviews
- Stack: Claude 3.5 Sonnet + LangGraph + Supabase (pgvector + Auth + DB) + FastAPI + React/Vite/Tailwind + Google OAuth

Full Plan: 8 Steps, 12–15 Hours

Step 1 - Validate and Fix Regulatory Corpus (1 hour)

What: Manually spot-check the 986 OGUC + PRC chunks for the 10–15 most critical articles. Verify that Art. 1.4.9 (Acta procedure), Art. 5.1.17 (permit requirements), and the Las Condes ZHR2 zone table are cleanly chunked and complete. Fix any broken chunks.
Why first: RAG quality is 80% determined by chunk quality. If the chunks are wrong, the agents will hallucinate regardless of how good the prompts are.
Output: A validated, corrected oguc_chunks.jsonl and prc_lascondes_chunks.jsonl ready for Supabase pgvector ingestion.

Step 2 — Define Evaluation Framework (30 min)

What: Create a simple eval set using the real Actas you already have. Take the Paine 2019 project parameters, run them through the AI, and compare the AI's observations against the 8 real DOM observations. Define three metrics: (1) did the AI catch the real violations? (2) did it hallucinate fake violations? (3) did it cite the correct articles?
Why: Without this, you cannot know when the prototype is good enough to demo. This is 30 minutes of setup that saves hours of guesswork.
Output: A eval/ground_truth.json file with the Paine project parameters and the 8 real observations as expected output.

Step 3 — Design Agent Prompts and Use Cases (1 hour)

What: Write the system prompts for all four agents. Define the exact input/output schema for each. Decide the two use cases the prototype will handle: (1) Edificación Nueva residential in ZHR2 (Las Condes), (2) Ampliación residential in ZHR2. These two cover 60%+ of real permit submissions.
Why before building: If you start coding without finalized prompts, you will refactor constantly. The prompts are the product logic - they should be designed, not improvised.
Output: A prompts/ directory with four system prompt files and an agent_schemas.py with Pydantic input/output models.

Step 4 — Build Supabase Schema (1 hour)

What: Set up the Supabase project with: (1) Google OAuth, (2) pgvector extension enabled, (3) four tables - users, projects, compliance_runs, observations - with RLS policies, (4) ingest the 986 regulatory chunks into the regulatory_chunks vector table.
Why: Everything else depends on this. The LangGraph agents need the vector store to exist before they can query it. Auth needs to be set up before the frontend can protect routes.
Output: A running Supabase project with schema, RLS, and populated vector store. A supabase/schema.sql file in the repo.

Step 5 — Build Backend: FastAPI + LangGraph Agents (4–5 hours)

What: Build the four-agent LangGraph graph and expose it via two FastAPI endpoints: POST /api/analyze (runs the full compliance check) and GET /api/runs/{run_id} (retrieves a past run). Implement LangGraph's Supabase checkpointer for state persistence.
Why this is the biggest time block: This is the core product. The graph state, agent transitions, error handling, and streaming response all live here.

Output: A working backend that accepts project parameters, runs the four agents, and returns a structured compliance report. Testable via curl or the eval script.
The project spec I already wrote covers this in full — Claude Code can scaffold it from that document.

Step 6 — Build Frontend: React + Human-in-the-Loop UX (3–4 hours)

What: Four screens only — no more:

Screen	What It Does
Login	Google OAuth via Supabase — one button
New Project	Form: project address, zone, use type, key metrics (constructibility, height, surface, parking)
Compliance Report	Streaming display of observations as they generate; each observation shows the article cited and the specific violation; a "Mark as Resolved" button per observation
Project History	List of past runs with status (Compliant / Observations / Rejected)

Why this scope: The human-in-the-loop element is the "Mark as Resolved" button - the architect reviews each observation, marks it resolved when corrected, and resubmits. This is the core product loop and it is achievable in 3–4 hours with Claude Code. There could be a back and forth chat also. There could also be a tool chat when the agent doesn't know the answer, calls an expert and the expert can type the answer into the chat and display that into the product.
Output: A working React frontend connected to the FastAPI backend, with Google Auth and streaming agent output.

Step 7 — Evaluate and Iterate (1–2 hours)

What: Run the eval framework against the Paine ground truth. Measure precision, recall, and hallucination rate. Fix the top 2–3 issues found. Run again.
Why: You need at least one evaluation cycle before the demo. Even a single pass will reveal the most obvious prompt failures.
Output: An eval report showing the prototype's performance on the ground truth set.

Step 8 — Demo Prep (30 min)

What: Prepare the Stepke project (12-floor building, Las Condes ZHR2) as the live demo case. Verify it runs end-to-end. Prepare a 2-minute walkthrough: input parameters → streaming compliance report → one observation marked as resolved.
Output: A rehearsed, reproducible demo.

Time Budget Summary

Step	Task	Hours
1	Validate regulatory corpus	1.0
2	Evaluation framework	0.5
3	Agent prompts + use cases	1.0
4	Supabase schema + ingestion	1.0
5	Backend: FastAPI + LangGraph	4.5
6	Frontend: React + UX	3.5
7	Evaluate + iterate	1.5
8	Demo prep	0.5
Total		13.5 hours